

Ejercicio 21: El Problema del Corte de Varas (Programación Dinámica)

Enunciado

Dada una vara que mide n centímetros y un vector que indica los precios de todas las varas de tamaño menor o igual que n , determinar cuál es el máximo valor que podemos conseguir cortando la vara y vendiendo los trozos obtenidos.

a) Ecuación en Recurrencias

Definimos la función óptima como:

- $corteVara(i)$: El beneficio máximo que se puede obtener a partir de una vara de longitud i (para $0 \leq i \leq n$).

Para calcular el valor óptimo para una vara de longitud i , debemos tomar una decisión sobre el tamaño del primer trozo a cortar, llamémoslo j ($1 \leq j \leq i$). El coste de esta decisión será el precio directo del trozo cortado ($precio[j]$) más el coste óptimo de procesar la longitud restante de la vara ($corteVara(i - j)$):

$$corteVara(i) = \begin{cases} 0 & \text{si } i = 0 \quad (\text{Caso Base}) \\ \max_{1 \leq j \leq i} \{ precio[j] + corteVara(i - j) \} & \text{si } i > 0 \end{cases}$$

Donde $precio[j]$ es el precio de venta de una vara de tamaño j (asumiendo indexación de 1 a n).

b) Algoritmo Recursivo con Memorización (Top-Down)

Este algoritmo resuelve el problema traduciendo directamente la relación de recurrencia en una función recursiva, utilizando un vector auxiliar `Memo` inicializado en -1 para evitar el cálculo redundante de subproblemas ya resueltos.

```
Función CorteVaraRecursivo(n, precio):
    // Crear un vector de tamaño (n + 1) para almacenar los resultados
    Memo[0..n] ← -1
    Memo[0] ← 0 // Caso base

    Retornar AuxiliarRecursivo(n, precio, Memo)

Función AuxiliarRecursivo(i, precio, Memo):
    // Si ya ha sido calculado previamente, devolvemos el valor guardado
    Si (Memo[i] ≠ -1) Entonces
        Retornar Memo[i]
    FinSi

    Max_Valor ← -INFINITO

    // Probar todos los posibles primeros cortes de longitud j
    Para j desde 1 hasta i hacer
```

```
        Si (Max_Valor < precio[j] + AuxiliarRecursivo(i - j, precio, Memo)) Entonces
```

```

Si (valor_Actual > max_valor) Entonces
    Max_Valor ← Valor_Actual
FinSi
FinPara

// Guardar el resultado en la memoria antes de retornar
Memo[i] ← Max_Valor
Retornar Max_Valor

```

c) Análisis de Eficiencia

- **Sin Memorización (Recursión Pura):**

- **Tiempo:** $\mathcal{O}(2^n)$. Al no guardar los resultados intermedios, cada llamada de tamaño i genera llamadas recursivas de tamaños $i - 1, i - 2, \dots, 0$. Esto crea un árbol de ejecución con un número de nodos exponencial que se duplica en cada nivel.
- **Espacio:** $\mathcal{O}(n)$ de memoria auxiliar, correspondiente a la profundidad máxima de la pila de llamadas del sistema.

- **Con Programación Dinámica (Top-Down con Memorización o Bottom-Up):**

- **Tiempo:** $\mathcal{O}(n^2)$. El número de subproblemas distintos a resolver es exactamente $n + 1$ (uno para cada longitud de 0 a n). Resolver cada subproblema de tamaño i requiere un bucle lineal que realiza exactamente i comparaciones. La suma de operaciones es:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \in \mathcal{O}(n^2)$$

- **Espacial:** $\mathcal{O}(n)$ para almacenar la tabla de estados intermedios de tamaño $n + 1$.

d) Solución Eficiente en Tiempo (Iterativa Bottom-Up con Reconstrucción)

Para una implementación real en exámenes de máxima nota, se prefiere la solución iterativa **de abajo hacia arriba** utilizando un vector.

Esta solución calcula primero los valores de las varas más cortas y los reutiliza. Además, incluye un vector auxiliar `corte[i]` que registra el tamaño del primer corte óptimo para poder indicarle al usuario **cómo cortar físicamente la vara** para alcanzar dicho valor máximo.

```

Algoritmo CorteVaraEficiente(n, precio)
// Entrada:
//   - n: Longitud total de la vara
//   - precio[1..n]: Precios de varas de tamaños 1 a n
// Salida:
//   - El beneficio máximo y los cortes exactos a realizar

// 1. Inicializar tablas de tamaño (n + 1)
Crear vector DP[0..n]
Crear vector corte[0..n]

DP[0] ← 0 // Caso base: vara de longitud 0 tiene beneficio 0

```

```

Para i ← 1 hasta n hacer
    Max_Valor ← -INFINITO
    Primer_Corte_Optimo ← -1

    // Probar todos los posibles primeros cortes j para una vara de tamaño i
    Para j ← 1 hasta i hacer
        Valor_Actual ← precio[j] + DP[i - j]

        Si (Valor_Actual > Max_Valor) Entonces
            Max_Valor ← Valor_Actual
            Primer_Corte_Optimo ← j
        FinSi
    FinPara

    DP[i] ← Max_Valor
    corte[i] ← Primer_Corte_Optimo
FinPara

// 3. Reconstrucción de la solución (Obtener los cortes óptimos)
Escribir "Beneficio Máximo Obtenido: ", DP[n]
Escribir "Cortes a realizar (longitudes):"

longitud_restante ← n
Mientras (longitud_restante > 0) hacer
    Escribir "- Pieza de tamaño: ", corte[longitud_restante]
    longitud_restante ← longitud_restante - corte[longitud_restante]
FinMientras

Retornar DP[n]
FinAlgoritmo

```